

Fourier transform, Filtering, Convolution

The **Fourier transform** allows the frequency content of signals to be analyzed. The *Numpy* library has an FFT package with a 2D variant for the image. The other basic tool for filtering signals is convolution. Both of these tools can be applied to the selection of particular frequencies or to the denoising of images.

1) Write a program to display an image and the modulus of its Fourier transform in 2D. What does the FFT 2D calculation function return? By default, what is the size of the output table? Can this size be changed? What is the `fftshift` function used for ? Locate high and low frequencies.

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

2) Observe the modulus of the 2D Fourier transform of several grayscale images: **cameraman.gif**, **bureau.gif**, **testpat1.tif**, **testpat2.tif**, **circuit.tif**, **ic.tif**, **ic45.tif**

Why is it better to view the TF module log?

What does an image rotation on the TF produce?

What does the TF(0,0) element represent? What is the relationship between this element, the average and the size of the image?

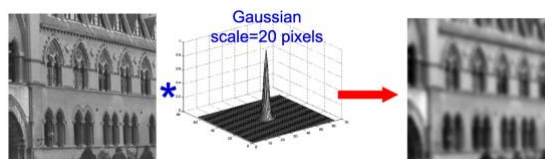
3) By applying a circular mask to the TF, remove some of the frequencies (low, high or medium), then apply the reverse TF to observe the changes in the image. Test several images: **lena.bmp**, **barbara.gif**, **testpat1.tif**...

Intuitively, where are the high and low frequencies located in an image?

See: https://docs.opencv.org/4.x/de/dbc/tutorial_py_fourier_transform.html

The other solution for image filtering is to use the **2D convolution** operator in the space domain. A square mask $M \times M$ with the coefficients of the filter $h(m,n)$, also called the *nucleus*, is moved over the entire image. The value of the central pixel $g(i,j)$ is determined by the sum of the values of the neighboring pixels, weighted by the coefficients of the filter

$$g(i, j) = \sum_{m=-M/2}^{M/2} \sum_{n=-M/2}^{M/2} h(m, n) f(i - m, j - n)$$



4) To start, apply a 3x3 size averaging filter on the image:

Start again with 11x11 size averaging filter. Evaluate the sharpness of the image with a quantitative indicator, such as the variance of the Laplacian, for example.

What effect do they have? What is the size of the output image? Evaluate the side effects.

5) Compare the result obtained by filtering the original image twice in a row with the 3x3 averaging (re-applying the convolution on the filtered image obtained), and the result obtained by filtering the original image with a 5x5 averaging device. We can realize a difference between the two resulting images. What is the best option in terms of calculations? What advantages does local processing bring compared to a global transformation such as the Fourier transform?

6) Subtract the image filtered by a 5x5 averaging from the original image. Visualize the result obtained, then its absolute value. Explain what this result represents. Binarize the image obtained by thresholding and display the binary image.

7) Try the following masks on the images at your disposal and deduce the filtering effect produced: low-pass, band-pass, high-pass, ...

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

Mask 1

-0,61	-2	-2,85	-2	-0,61
-2	1,61	3,85	1,61	-2
-2,85	3,85	8	3,85	-2,85
-2	1,61	3,85	1,61	-2
-0,61	-2	-2,85	-2	-0,61

Mask 2

-0,38	1	1,85	1	-0,38
1	-2,61	-4,85	-2,61	1
1,85	-4,85	16	-4,85	1,85
1	-2,61	-4,85	-2,61	1
-0,38	1	1,85	1	-0,38

Mask 3

-1	-1	-1
-1	8	-1
-1	-1	-1

Mask 4

-1	0	1
-2	0	2
-1	0	1

Mask 5

-1	-2	-1
0	0	0
1	2	1

Mask 6

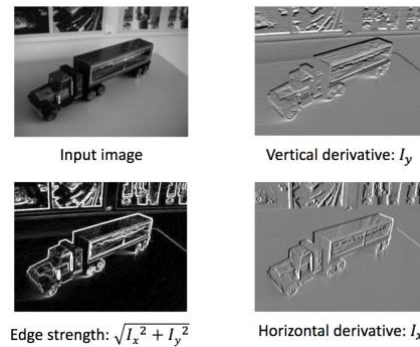
Add the image filtering results by masks 1, 2, and 3. Conclusion.

Subtract from the image averaged by the 3x3 mask, the hundredth of the result of the filtering by the mask 4. What effect does this combination have?

8) Apply masks 5 and 6 to image **carre.png**. Visualize the absolute value of the sum of the resulting images from the filters by masks 5 and 6. What does it represent?

Verify that Mask 5 is equivalent to the combination of one-dimensional filters $[1 \ 2 \ 1]^T * [-1 \ 0 \ 1]$. What do these two filters produce individually?

What mathematical relationship connects masks 5 and 6?



See: https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

9) Construct a **Gaussian pyramid** from an image by recursively applying Gaussian filtering. How do you calculate the size of the Gaussian filter based on the *sigma value* ?

Denoising

The acquisition of digital images introduces different sources of noise related to the sensor (defective pixels, dark current, etc.), to quantization or digital transmission, etc. We will simulate the presence of noise in the images and study some filtering methods to reduce noise.

Four types of noise have been added to the **cameraman** image. The following images are at your disposal:



- Gaussian or impulse **additive noise** : $I = I + B$

Cam_Gaus01: Added centered white noise of variance 0.01

Cam_Gaus005: Added centered white noise with variance 0.005

Cam_Imp01 : N random white or black pixels, $N = 0.01 * \text{prod}(\text{size}(I))$

Cam_Imp05 : N random white or black pixels, $N = 0.05 * \text{prod}(\text{size}(I))$

- Multiplicative **noise** (speckle): $I = I * B$

Cam_Spe01 : Uniformly distributed centered B of variance 0.01

Cam_Spe04 : Evenly distributed centered B of variance 0.04

- Poisson Noise : **Cam_Poisson**

10) **Calculation of the SNR (Signal to Noise ratio)**: Display the **original** cameraman.bmp image, its noisy version **Cam_Gaus005.bmp** and that of the noise by subtraction. View their respective histograms. Calculate the mean and standard deviation of each image (original and noisy). Check that the Gaussian noise added to the **cameraman image** has a variance of 0.005, if we relate its standard deviation to the range of 255 gray levels, by the calculation **varGaus = (std_Bruit/255)^2**. Calculate the SNR (Signal to Noise Ratio) in decibels: **$20 * \log(\text{std_Im}^2 / \text{std_Bruit}^2)$**

11) **Noise reduction by averaging**: create 4 images by adding a random Gaussian noise of zero mean and variance 0.005 to the **cameraman image**. View these four achievements and check that they are not identical. Calculate the average image of these 4 achievements. Display the difference of the original image with one of the 4 creations and the one with the average image on the same figure.

Calculate the variance of these two differences. What conclusion do you draw from this?

By repeating the manipulation with 8 realizations, verify that the averaging causes a reduction in the noise variance by a factor N , with N the number of averaged realizations.

The constraint of the averaging method is that it requires the time to make several successive acquisitions of the same scene without moving.

12) **Convolutional linear filtering** : compare the effect of the 3x3 averaging filter on the different types of noise (Gaussian or impulse additive, multiplicative, etc.), display 4 proposed noisy images and the 4 filtered images corresponding to them in the same figure. What do you conclude from this?

13) **Median filtering** consists of keeping the median value of each neighborhood to assign it to the central pixel. The size of the neighborhood can vary. This filter is part of the family of nonlinear order filters (it cannot be done by convolution). Apply the middle filter to the previous 4 types of noise. Compare the results on the same figure. What is the main advantage of this filter compared to an averager? What type of noise is the middle filter particularly effective at?

To go further... in PEA

You can read up on other filtering methods, such as adaptive filters, bilateral filters, etc., and test them on images.

See: https://docs.opencv.org/4.x/d5/d69/tutorial_py_non_local_means.html